

Text Mining 2

Anthony Kamau

Contents

Word Embedding	1
Load data set	2
Tokenize data frame	3
Remove stop words	3
Vocabulary of unique terms	4
Token Co-occurrence Matrix (TCM)	4
Train word vectors	5
Combining and extracting word vectors	5
Finding most similar words	5
More word vector manipulations	6
Dictionary-based Sentiment Analysis	7
Load dataset	7
Tidyttext package 4 general purpose lexicons	7
Download the labMT dictionary	7
Tokenization	8
Stop words removal	8
Finding sentiment scores	9
Average sentiment score for each review	9

```
# install.packages(c("text2vec"))  
library(tidyverse)  
library(tidyttext)  
library(text2vec)
```

Word Embedding

To work with text, we need to transform it into numerical values. One way is to count the words and weight their frequency (e.g. with TF-IDF). Another method is word embedding. Word embedding techniques such as word2vec and GloVe use neural networks to construct word vectors. With these techniques, words in similar contexts are represented by similar numerical vectors.

Load data set

We will use the Harry Potter books as our data. Installing the `harrypotter` package from github using `remotes`.

```
# install.packages("remotes")
remotes::install_github("bradleyboehmke/harrypotter")
```

```
## Skipping install of 'harrypotter' from a github remote, the SHA1 (51f71461) has not changed since last
## Use 'force = TRUE' to force installation
```

```
library(harrypotter) # Not to be confused with the CRAN palettes package
```

It is a data set with all Harry Potter books, containing the full texts of the first seven Harry Potter books, Each text is in a character vector with each element representing a single chapter. It is provided from the `harrypotter` package written by Bradley Boehmke.

- **Philosophers_stone**: Harry Potter and the Philosophers Stone, published in 1997
- **Chamber_of_secrets**: Harry Potter and the Chamber of Secrets, published in 1998
- **Prisoner_of_azkaban**: Harry Potter and the Prisoner of Azkaban, published in 1999
- **Goblet_of_fire**: Harry Potter and the Goblet of Fire, published in 2000
- **Order_of_the_phoenix**: Harry Potter and the Order of the Phoenix, published in 2003
- **Half_blood_prince**: Harry Potter and the Half-Blood Prince, published in 2005
- **Deathly_hallows**: Harry Potter and the Deathly Hallows, published in 2007

Load the first seven novels in the Harry Potter series

```
hp_books <- c("philosophers_stone", "chamber_of_secrets",
             "prisoner_of_azkaban", "goblet_of_fire",
             "order_of_the_phoenix", "half_blood_prince",
             "deathly_hallows")

hp_words <- list(
  philosophers_stone,
  chamber_of_secrets,
  prisoner_of_azkaban,
  goblet_of_fire,
  order_of_the_phoenix,
  half_blood_prince,
  deathly_hallows
) |>
  # names each list element
  set_names(hp_books) |>
  # converts each book to a data frame and merge into a single data frame
  map_df(as_tibble, .id = "book") |>
  # converts book to a factor
  mutate(book = factor(book, levels = hp_books)) |>
  # removes empty chapters
```

```

filter(!is.na(value)) |>
# creates a chapter id column
group_by(book) |>
mutate(chapter = row_number(book))

# head(hp_words)

```

Tokenize data frame

We'll use `unnest_tokens` function from the `tidytext` package to tokenize the data frame

```

hp_words <- hp_words |>
  unnest_tokens(word, value)

head(hp_words)

```

```

## # A tibble: 6 x 2
## # Groups:   book [1]
##   book          word
##   <fct>         <chr>
## 1 philosophers_stone the
## 2 philosophers_stone boy
## 3 philosophers_stone who
## 4 philosophers_stone lived
## 5 philosophers_stone mr
## 6 philosophers_stone and

```

Remove stop words

We remove stop words from tokenized data frame

```

hp_words <- hp_words |>
# remove stop words
anti_join(stop_words)

```

```
## Joining with 'by = join_by(word)'
```

```
head(hp_words)
```

```

## # A tibble: 6 x 2
## # Groups:   book [1]
##   book          word
##   <fct>         <chr>
## 1 philosophers_stone boy
## 2 philosophers_stone lived
## 3 philosophers_stone dursley
## 4 philosophers_stone privet
## 5 philosophers_stone drive
## 6 philosophers_stone proud

```

Vocabulary of unique terms

Creating a vocabulary of unique terms using the `create_vocabulary` function from the `text2vec` package and removing the words that appear less than 5 times.

```
# make it a list (iterator)
hp_words_ls <- list(hp_words$word)

# create index-tokens
it <- itoken(hp_words_ls, progressbar = FALSE)

# collects unique terms
hp_vocab <- create_vocabulary(it)

# filters the infrequent terms (number of occurrence is less than 5)
hp_vocab <- prune_vocabulary(hp_vocab, term_count_min = 5)

# show the resulting vocabulary object (formatting it with datatable)
hp_vocab
```

```
## Number of docs: 1
## 0 stopwords: ...
## ngram_min = 1; ngram_max = 1
## Vocabulary:
##           term term_count doc_count
##           <char>      <int>    <int>
## 1:           10          5         1
## 2:        aaaah          5         1
## 3:        aargh          5         1
## 4:       abysmal          5         1
## 5:         acts          5         1
## ---
## 8624:      looked       2344         1
## 8625: dumbledore       2873         1
## 8626:  hermione       4912         1
## 8627:         ron       5750         1
## 8628:       harry      16557         1
```

```
# # We've just created word counts, that's all the vocabulary object is!
```

Token Co-occurrence Matrix (TCM)

Creating a token co-occurrence matrix (TCM). We first apply `vocab_vectorizer` function to transform the list of tokens into a vector space. Then, use `create_tcm` function to create a TCM with the window of 5 for context words.

```
# maps words to indices
vectorizer <- vocab_vectorizer(hp_vocab)

# use window of 5 for context words
hp_tcm <- create_tcm(it, vectorizer, skip_grams_window = 5)
```

```
# Note that such a matrix will be extremely sparse.
# Most words do not go with other words in the grand scheme of things. So when
# they do, it usually matters.
```

Train word vectors

Fit the word vectors on our data set, with embedding size (rank variable) equal to 50, and the maximum number of co-occurrences equal to 10. Train word vectors in 20 iterations.

```
glove      <- GlobalVectors$new(rank = 50, x_max = 10)
hp_wv_main <- glove$fit_transform(hp_tcm, n_iter = 20, convergence_tol = 0.001)
```

Combining and extracting word vectors

The GloVe model learns two sets of word vectors: main and context. Essentially they are the same since the model is symmetric. In general combining the two sets of word vectors leads to higher quality embeddings. The best practice is to combine both the main word vectors and the context word vectors into one matrix. Extract the word vectors and save the summation of them.

```
# extract context word vector
hp_wv_context <- glove$components

# check the dimension for both matrices
dim(hp_wv_main); dim(hp_wv_context)
```

```
## [1] 8628  50
```

```
## [1]   50 8628
```

```
# Either word-vectors matrices could work, but the developers of the technique
# suggest the sum/mean may work better

# transpose one matrix to perform matrix addition
hp_word_vectors <- hp_wv_main + t(hp_wv_context)
```

Finding most similar words

Find the most similar 10 words to each of the words: “harry”, “death”, and “love”

HARRY

```
# extract the row of "harry"
harry <- hp_word_vectors["harry", , drop = FALSE]

# calculates pairwise similarities between "harry" and the rest of words
cos_sim_harry <- sim2(x = hp_word_vectors, y = harry, method = "cosine", norm = "l2")

# the top 10 words with the highest similarities
head(sort(cos_sim_harry[,1], decreasing = T), 10)
```

```
##      harry      ron  hermione  moment      time  looked  quickly  harry's
## 1.0000000 0.8617774 0.8577938 0.8384842 0.8070126 0.7775984 0.7611351 0.7249779
##      left      told
## 0.7233178 0.7033968
```

DEATH

```
# extract the row of "death"
death <- hp_word_vectors["death", , drop = FALSE]

# calculates pairwise similarities between "harry" and the rest of words
cos_sim_death <- sim2(x = hp_word_vectors, y = death, method = "cosine", norm = "l2")

# the top 10 words with the highest similarities
head(sort(cos_sim_death[,1], decreasing = T), 10)
```

```
##      death  eaters  eater voldemort  fight  people  killed  escaped
## 1.0000000 0.9412572 0.8907935 0.6423951 0.6098150 0.6041998 0.5688789 0.5455654
##      life      kill
## 0.5311815 0.5278262
```

LOVE

```
# extract the row of "love"
love <- hp_word_vectors["love", , drop = FALSE]

# calculates pairwise similarities between "harry" and the rest of words
cos_sim_love <- sim2(x = hp_word_vectors, y = love, method = "cosine", norm = "l2")

# top 10 words with the highest similarities
head(sort(cos_sim_love[,1], decreasing = T), 10)
```

```
##      love      lie      potion  potions  wonderful      bet
## 1.0000000 0.5893449 0.5495199 0.5436379 0.5300657 0.4919974
##      lucky desperately      meant      steal
## 0.4909327 0.4901501 0.4866008 0.4717936
```

More word vector manipulations

Play around with the word vectors. For example, add the word vector of “harry” with the word vector of “love” and subtract them from the word vector of “death” and find the top terms in the result.

```
# add/subtract word vectors
test <- harry + love - death

# calculates pairwise similarities between "harry" and the rest of words
cos_sim_test <- sim2(x = hp_word_vectors, y = test, method = "cosine", norm = "l2")

# top 10 words with the highest similarities
head(sort(cos_sim_test[,1], decreasing = T), 10)
```

```
##      ron      harry      dunno  hermione      yeah  thinking  quickly      time
## 0.6512234 0.6502382 0.6420924 0.6386039 0.5751631 0.5734916 0.5671082 0.5418939
##      fred  slughorn
## 0.5314027 0.5180239
```

Dictionary-based Sentiment Analysis

We are going to use the data set `movie_review`. This is a data set with 5,000 IMDB movie reviews available from the `text2vec` package, labeled according to their IMDB rating. The sentiment of the reviews is binary, meaning an IMDB rating < 5 results in a sentiment score of 0, and a rating ≥ 7 has a sentiment score of 1. No individual movie has more than 30 reviews.

Load dataset

```
data("movie_review")

movie_review <- as_tibble(movie_review)

head(movie_review)

## # A tibble: 6 x 3
##   id      sentiment review
##   <chr>      <int> <chr>
## 1 5814_8          1 "With all this stuff going down at the moment with MJ i've s~
## 2 2381_9          1 "\"The Classic War of the Worlds\"" by Timothy Hines is a~
## 3 7759_3          0 "The film starts with a manager (Nicholas Bell) giving welco~
## 4 3630_4          0 "It must be assumed that those who praised this film (\""th~
## 5 9495_8          1 "Superbly trashy and wondrously unpretentious 80's exploitat~
## 6 8196_8          1 "I dont know why people think this is such a bad movie. Its ~
```

Tidyttext package 4 general purpose lexicons

The `tidyttext` package contains 4 general purpose lexicons in the sentiments dataset.

- **afinn**: list of English words rated for valence between -5 and +5
- **bing**: list of positive and negative sentiment
- **nrc**: list of English words and their associations with 8 emotions (anger, fear, anticipation, trust, surprise, sadness, joy, and disgust) and 2 sentiments (negative and positive); binary
- **loughran**: list of sentiment words for accounting and finance by category (Negative, Positive, Uncertainty, Litigious, Strong Modal, Weak Modal, Constraining)

We are going to use *labMT dictionary* (Dodds' et al. 2011), one of the best dictionaries for sentiment analysis (see e.g. [this paper](#).)

Download the labMT dictionary

```
labMT_url <- "https://github.com/andyreagan/sentidict/raw/refs/heads/main/data/LabMT/labMT2english.txt."
labMT <- readr::read_delim(labMT_url, delim = "\t") |> select(word, value = happens)
```

Tokenization

Use `unnest_tokens` function from `tidytext` package to break the `movie_review` data set into individual tokens.

```
# tokenize the reviews
tidy_review <-
  movie_review |>
  unnest_tokens(word, review)

head(tidy_review)
```

```
## # A tibble: 6 x 3
##   id      sentiment word
##   <chr>      <int> <chr>
## 1 5814_8          1 with
## 2 5814_8          1 all
## 3 5814_8          1 this
## 4 5814_8          1 stuff
## 5 5814_8          1 going
## 6 5814_8          1 down
```

Stop words removal

Remove the stop words from the tokenized data frame

```
tidy_review <- tidy_review |>
  # remove stop words
  anti_join(stop_words)
```

```
## Joining with 'by = join_by(word)'
```

```
head(tidy_review)
```

```
## # A tibble: 6 x 3
##   id      sentiment word
##   <chr>      <int> <chr>
## 1 5814_8          1 stuff
## 2 5814_8          1 moment
## 3 5814_8          1 mj
## 4 5814_8          1 started
## 5 5814_8          1 listening
## 6 5814_8          1 music
```

Finding sentiment scores

Using the `inner_join` function to find a sentiment score for each of the tokenized review words using the `labMT` dictionary

```
review_sentiment <-  
  tidy_review |>  
  inner_join(labMT)
```

```
## Joining with 'by = join_by(word)'
```

```
head(review_sentiment)
```

```
## # A tibble: 6 x 4  
##   id      sentiment word      value  
##   <chr>      <int> <chr>    <dbl>  
## 1 5814_8          1 stuff     5.58  
## 2 5814_8          1 moment    5.68  
## 3 5814_8          1 mj        5.06  
## 4 5814_8          1 started    6  
## 5 5814_8          1 listening  6.28  
## 6 5814_8          1 music     8.02
```

Average sentiment score for each review

Calculating the average sentiment score for each review. Comment on the three most positive and negative reviews (i.e., which has the highest and lowest average sentiment score)?

```
sorted_review_sentiment <-  
  review_sentiment |>  
  # group by review id  
  group_by(id) |>  
  # compute the average sentiment score  
  summarize(average_sentiment = mean(value),  
            sentiment = mean(sentiment)) |>  
  # arrange the average sentiment score in descending order  
  arrange(desc(average_sentiment))
```

```
sorted_review_sentiment
```

```
## # A tibble: 5,000 x 3  
##   id      average_sentiment sentiment  
##   <chr>          <dbl>      <dbl>  
## 1 12421_10        6.99          1  
## 2 6601_4          6.94          0  
## 3 10780_10        6.90          1  
## 4 2684_10         6.85          1  
## 5 8714_10         6.83          1  
## 6 4518_9          6.78          1  
## 7 226_10          6.75          1  
## 8 11841_9         6.74          1
```

```
## 9 2642_10          6.72      1
## 10 9144_7          6.72      1
## # i 4,990 more rows
```

HAPPIEST REVIEWS

```
# filter happiest reviews
movie_review |>
  filter(id %in% head(sorted_review_sentiment$id, 3)) |>
  pull(review)
```

```
## [1] "I can see why Laurel and Hardy purists might be offended by this rather gentle 're-enactment', but it's not as bad as you think."
## [2] "Far by my most second favourite cartoon Spielberg did, after Animaniacs. Even if the ratings were low, it's a masterpiece."
## [3] "Very smart, sometimes shocking, I just love it. It shoved one more side of David's brilliant talent into the world."
```

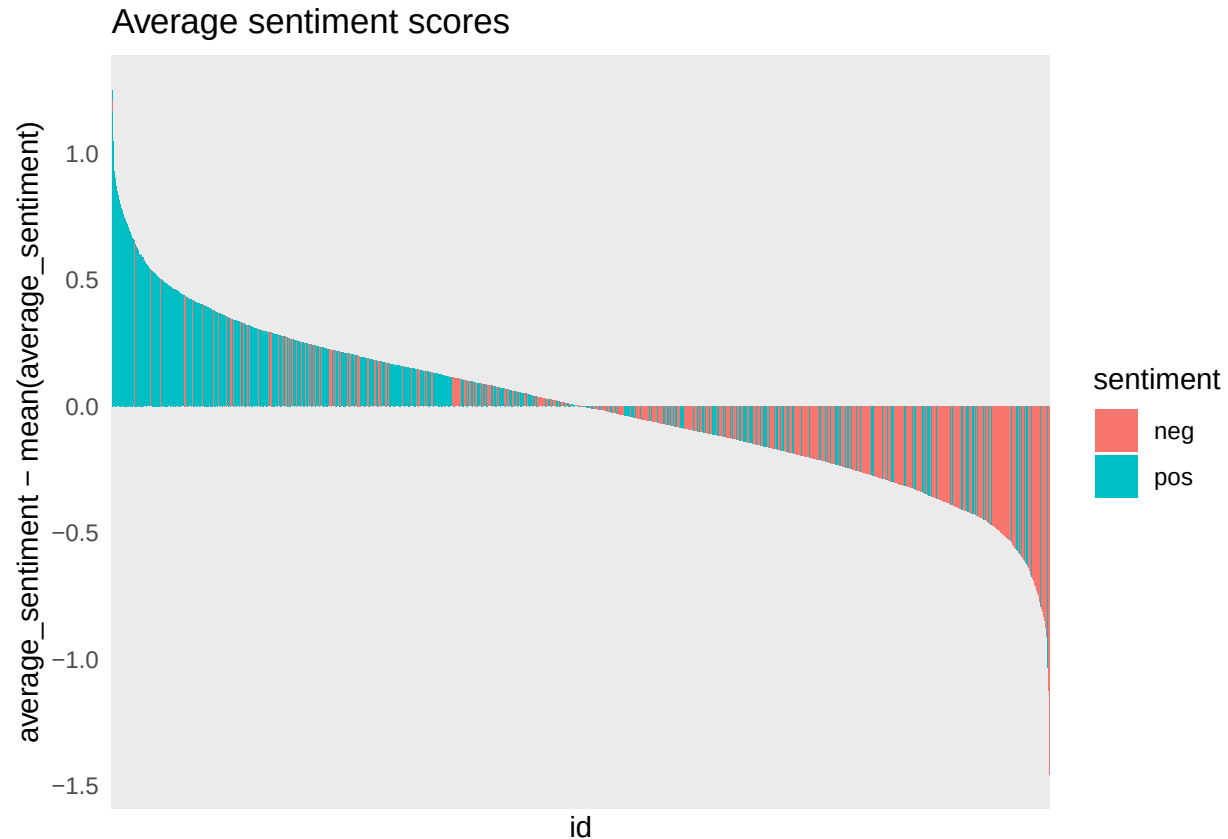
SADDEST REVIEWS

```
# filter saddest reviews
movie_review |>
  filter(id %in% tail(sorted_review_sentiment$id, 3)) |>
  pull(review)
```

```
## [1] "This isn't the worst movie I've ever seen, but I really can't recall when I've seen a worse one."
## [2] "This tear-teaser, written by Steve Martin himself, is so unbelievably bad, it makes you sick to your stomach."
## [3] "The characters are unlikeable and the script is awful. It's a waste of the talents of Deneuve and the director."
```

Bar chart of these average sentiment scores across the ids. Color used to show the original sentiment.

```
sorted_review_sentiment |>
  # create the coloring variable
  mutate(sentiment = ifelse(sentiment > 0, "pos", "neg")) |>
  # reorder `id` such that average sentiment scores are presented in descending order.
  ggplot(aes(
    x = reorder(id, -average_sentiment),
    y = average_sentiment - mean(average_sentiment),
    fill = sentiment
  )) +
  # plot the bar chart using `geom_col`
  geom_col() +
  labs(x = "id", title = "Average sentiment scores") +
  theme_minimal() +
  theme(axis.text.x=element_blank())
```



Creating a `predicted_sentiment` column, such that `_average` score higher than 5.75 is positive = 1 and average score lower than or equal to 5.75 is negative = 0. Then, using a confusion matrix to compare this `predicted_sentiment` to the original sentiment.

```
review_sentiment_agg <-
  sorted_review_sentiment |>
  # Dichotomize average sentiment score to match with the original sentiment scores
  # 1: scores higher than zero; 0: scores lower than or equal to zero
  mutate(predicted_sentiment = ifelse(average_sentiment > 5.75, 1, 0),)

## not too bad
table(
  true = review_sentiment_agg$sentiment,
  predicted = review_sentiment_agg$predicted_sentiment
)
```

```
##      predicted
## true    0    1
##    0 1729  754
##    1  864 1653
```

```
# Accuracy = 1653+1729/5000 = 67.64%
```

Following the same steps, but only for the reviews in the top or bottom 25% of the distribution of average sentiment (there are likely many neutral reviews). Accessing our accuracy and whether our results improve.

```
summary(sorted_review_sentiment$average_sentiment)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    4.280   5.528   5.736   5.736   5.945   6.987
```

```
review_sentiment_agg <- sorted_review_sentiment |>
  filter((average_sentiment > 5.945) | (average_sentiment < 5.528)) |>
  # Dichotomize average sentiment score to match with the original sentiment scores
  # 1: scores higher than zero; 0: scores lower than or equal to zero
  mutate(predicted_sentiment = ifelse(average_sentiment > 5.75, 1, 0),)
```

```
## not too bad
```

```
table(
  true = review_sentiment_agg$sentiment,
  predicted = review_sentiment_agg$predicted_sentiment
)
```

```
##      predicted
## true    0    1
##      0 935 259
##      1 317 990
```

```
# Accuracy = (935+990)/2501 = 77.0%
# Our accuracy increases considerably
```